
Data Extraction, Data Processing and Knowledge Extraction

Final Report - Big Data on Cloud, Modular Assignment 1

Every number in this report was produced by the pipeline code in this repository (see `reports/results/`) and is copied in by `reports/build_report.py`.

1. Introduction

A retailer's transaction log hides how customers buy: which products go together, who the valuable customers are and which records are unreliable. This project builds **one connected pipeline** that extracts the UCI Online Retail data, cleans it with **AWS Glue**, simulates **Change Data Capture (CDC)** with versioned snapshots, makes the data queryable in **Amazon Athena**, loads it into **Amazon Redshift**, mines purchase patterns with the **Apriori** algorithm, detects noise and outliers, measures similarity between customers and products, and documents the findings.

2. Problem statement

Design and implement a scalable data pipeline to extract, process and analyse transactional data for discovering customer purchasing patterns using AWS services and data mining techniques. (assignment PDF)

3. Dataset description

UCI **Online Retail** dataset: transactions of a UK-based online gift-ware retailer between **2010-12-01** and **2011-12-09**. Many customers are wholesalers.

Item	Value
Rows x columns	541,909 x 8
Columns	InvoiceNo (text), StockCode (text), Description (text), Quantity (integer), InvoiceDate (timestamp), UnitPrice (decimal), CustomerID (integer, nullable), Country (text)
Missing values	Description 1,454; CustomerID 135,080 (24.93% of rows); no other column
Exact duplicate rows	5,268
Distinct invoices / customers / products / countries	25,900 / 4,372 / 4,070 / 38
Cancellation invoices (C . . .) / adjustment invoices (A . . .)	9,288 rows / 3 rows
Rows with Quantity <= 0 / UnitPrice <= 0	10,624 / 2,517
Stock codes that are not product codes (POST, DOT, M, BANK CHARGES, gift_* ...)	2,995 rows
Lines per invoice	mean 20.92, max 1,114
Structure	one row = one product line of an invoice; an invoice belongs to exactly one customer (0 invoices with several customers); (InvoiceNo, StockCode) repeats in 10,684 rows, so it is not a unique key; 650 stock codes carry more than one description spelling
Geography	91.43% of rows are from the United Kingdom

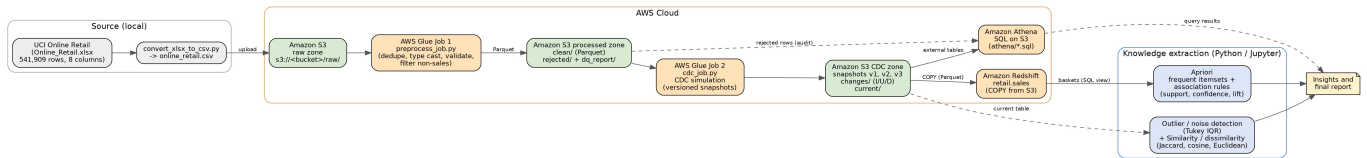
4. Methodology

- Glue job 1** reads the raw CSV, normalises and type-casts it, removes duplicates, rejects invalid rows (with a reason) and writes clean Parquet.
- Glue job 2** builds three versioned snapshots, derives the change log between them (insert / update / delete) and replays it to obtain the *current* table.
- Athena** queries the current table directly on S3; **Redshift** receives it with `COPY`.
- Apriori** runs on baskets (invoices) read from the warehouse layer; **outlier/noise detection** (Tukey IQR) and

similarity measures (Jaccard, cosine, Euclidean) run in a notebook.

5. Results are compiled in this report. Detailed design decisions are in `docs/process_documentation.md`.

5. AWS architecture



Raw CSV lands in an S3 *raw zone*; Glue job 1 writes the *processed zone*; Glue job 2 writes the *CDC zone* (snapshots, change log, current). Athena reads S3 in place (schema-on-read); Redshift loads the same Parquet with `COPY`. Python/Jupyter reads baskets from Redshift (or from the same Parquet when run locally) for mining. Only services that are actually used appear in the diagram.

6. Data preprocessing (AWS Glue)

Rules, in the order applied; the first matching rule gives the reject reason:

Step	Rule	Why
Normalise	trim text, upper-case <code>StockCode/Description</code> , empty string -> NULL	consistent keys, avoids "85123a" vs "85123A"
Type conversion	quantity -> int, price -> decimal, date -> timestamp, <code>CustomerID</code> -> int	correct types for SQL and Parquet
Duplicates	drop rows identical in all 8 columns	repeated records distort counts and support
Reject: cancellation / adjustment	<code>InvoiceNo</code> is not exactly 6 digits (<code>C...</code> , <code>A...</code>)	not sales
Reject: invalid quantity	quantity NULL or <= 0	not a sale
Reject: invalid price	price NULL or <= 0	free/test/adjustment rows
Reject: non-product code	<code>StockCode</code> not 5 digits + optional letters	postage, fees, manual entries are not products and would create meaningless rules
Keep	rows without <code>CustomerID</code>	valid baskets for Apriori; customer-level queries filter <code>customer_id IS NOT NULL</code>
Derive	<code>line_total = quantity x unit_price</code> ; <code>line_id = invoice + line number</code>	revenue; unique key needed for CDC

Result (from the job's data-quality report):

Count	Rows
Raw rows	541,909
Exact duplicates removed	5,268
Rejected: cancellation / adjustment	9,254
Rejected: invalid quantity	1,336
Rejected: invalid price	1,174
Rejected: non-product stock code	2,373
Clean rows	522,504 (96.42% of raw)
Clean invoices / customers / products	19,773 / 4,334 / 3,791
Clean rows without CustomerID	131,354

The job asserts that `raw = duplicates + rejected + clean` (no row is lost silently) and that no price loses precision when stored with two decimals. Rejected rows are kept in `rejected/` for audit.

7. CDC simulation (versioned snapshots)

This is a simulation, not a production real-time CDC system (no database log reader, no streaming, no DMS). The UCI file is a static dump, so it has no real update/delete history; the snapshots are therefore built as follows:

- **v1** - all clean lines up to 2011-09-30 (initial snapshot)
- **v2** - v1 + synthetic corrections/voids + the October 2011 lines

- **v3** - v2 + synthetic corrections/voids + the November-December 2011 lines

Synthetic changes per transition are chosen with a deterministic hash of `line_id` (reproducible, no random seed): 0.5% of lines get a quantity correction (+1 unit), 0.2% are deleted. Changes are then **identified** by comparing snapshots on the key `line_id` using a SHA-256 row hash: key only in the new snapshot = **Insert**, key only in the old = **Delete**, key in both with a different hash = **Update**. Replaying the change log on v1 yields the current table, which is verified to equal v3.

Snapshot	Lines
v1	357,721
v2	415,310
v3 = current	520,962 (replay equals v3: True)

Transition	Inserts	Updates	Deletes
v1 -> v2	58,305	1,821	716
v2 -> v3	106,478	2,010	826

Consequence: the current table has 520,962 lines, i.e. 1,542 fewer than the clean table (1,542 synthetic deletes; 3,831 synthetic quantity corrections). All later analysis uses the **current** table. Sample before/after rows are in `reports/results/cdc_sample_output.txt`.

8. Athena analysis

Queries are in `athena/*.sql`. They were validated against the same schema and Parquet files on a local DuckDB engine (Athena itself needs an AWS account); the sample outputs are in `athena/sample_outputs/`.

Headline numbers (Q: total transactions, customers, products):

Metric	Value
Line items	520,962
Transactions (invoices)	19,770
Customers	4,334
Products	3,791
Units sold	5,550,160
Revenue	10,231,034.78

Revenue by month (`02_revenue_analysis.sql` Q1):

month	transactions	revenue	avg_transaction_value
2010-12	1,550	772,986.46	498.70
2011-01	1,081	669,940.12	619.74
2011-02	1,093	506,057.45	463.00
2011-03	1,440	687,769.89	477.62
2011-04	1,235	514,262.40	416.41
2011-05	1,668	739,216.04	443.18
2011-06	1,524	736,341.23	483.16
2011-07	1,451	687,130.13	473.56
2011-08	1,339	722,239.41	539.39
2011-09	1,818	1,026,494.86	564.63
2011-10	2,004	1,101,996.47	549.90
2011-11	2,751	1,452,112.69	527.85
2011-12	816	614,487.63	753.05

Top countries by revenue (Q2):

country	transactions	revenue	revenue_share_pct
United Kingdom	17,898	8,713,513.55	85.17
Netherlands	93	282,741.46	2.76
EIRE	282	270,077.77	2.64
Germany	443	204,723.39	2.00

France	382	184,173.54	1.80
Australia	56	136,536.03	1.33
Spain	88	55,520.25	0.540
Switzerland	50	52,962.88	0.520
Japan	19	37,421.32	0.370
Sweden	34	36,836.93	0.360

Top customers by revenue (03_customer_analysis.sql Q1):

customer_id	transactions	units	revenue
14,646	72	196,594.00	277,989.04
18,102	60	64,127.00	259,666.08
17,450	46	69,954.00	194,289.07
16,446	2	80,997.00	168,472.50
14,911	198	80,000.00	135,914.43
12,415	20	76,440.00	123,030.15
14,156	54	57,539.00	116,070.32
17,511	31	64,427.00	90,931.64
12,346	1	74,215.00	77,183.60
16,029	62	40,107.00	72,708.09

Purchase-frequency structure (Q2):

invoices_per_customer	customers	revenue
1 (one-time)	1,505	625,682.43
2-3	1,333	1,345,397.96
4-9	1,110	2,296,363.67
10+	386	4,453,214.57

Top products by revenue (04_product_analysis.sql Q2):

stock_code	description	revenue	units_sold
22423	REGENCY CAKESTAND 3 TIER	173,288.72	13,784.00
23843	PAPER CRAFT , LITTLE BIRDIE	168,469.60	80,995.00
85123A	WHITE HANGING HEART T-LIGHT HOLDER	106,331.72	37,904.00
47566	PARTY BUNTING	99,466.81	18,285.00
85099B	JUMBO BAG RED RETROSPOT	94,135.83	48,358.00
23166	MEDIUM CERAMIC TOP STORAGE JAR	81,703.42	78,035.00
23084	RABBIT NIGHT LIGHT	66,881.90	30,744.00
22086	PAPER CHAIN KIT 50'S CHRISTMAS	64,886.99	19,333.00
84879	ASSORTED COLOUR BIRD ORNAMENT	58,880.06	36,334.00
79321	CHILLI LIGHTS	54,045.36	10,292.00

Basket-size distribution (05_transaction_analysis.sql Q2):

items_in_basket	transactions
1	1,503
2-5	2,857
6-10	3,012
11-25	6,625
26-50	3,684
51+	2,089

Mean transaction value is 517.50 with 26.09 distinct items on average; 387 of 3,791 products (10.21%) appear in fewer than 5 transactions.

9. Redshift implementation

- **Table** `retail.sales` (10 columns, same order as the Parquet files because `COPY` from Parquet maps by position); `DISTKEY(invoice_no)` keeps the lines of a basket on one slice, `SORTKEY(invoice_date)` speeds up time-range queries.
- **Load:** `COPY retail.sales FROM 's3://<bucket>/cdc/current/' IAM_ROLE '<role>' FORMAT AS PARQUET`, preceded by `TRUNCATE` so reloads are idempotent; `ANALYZE` afterwards.
- **Views** `retail.v_basket_items` (distinct invoice/product pairs) and `retail.v_product_names` feed the Apriori step.
- **Validation queries** (`redshift/03_validation_queries.sql`): totals must equal the Athena result, no invalid values, `line_id` unique.
- The portable parts of these queries were executed on DuckDB with the same data and gave: 520,962 lines, 19,770 transactions, 4,334 customers, 3,791 products, 0 invalid rows, 0 duplicate `line_id`. The DDL (`DISTKEY`, `SORTKEY`) and `COPY ... IAM_ROLE` are Redshift-specific and **were not executed** here.

10. Apriori results

Basket preparation. 19,770 baskets (invoices); the 18,267 with at least 2 distinct products are used (a one-item basket cannot form a co-purchase), over 3,786 products identified by `StockCode`. Support is relative to these 18,267 baskets.

Thresholds. `min_support = 0.01` (an itemset must appear in at least 183 baskets), `min_confidence = 0.5`, `min_lift = 1.5`. They were chosen from this sensitivity table: 5% or 3% support finds almost no pairs (the catalogue is long-tailed), 1% is the lowest level that still gives a manageable number of statistically meaningful patterns, and it runs in seconds. Lift > 1.5 removes pairs that co-occur only because both are popular.

min_support	min_count	frequent_itemsets	size_1	size_2	size_3plus	rules_conf>=0.5	seconds
0.050	914	46	46	0	0	0	0.810
0.030	549	165	148	17	0	16	1.73
0.020	366	469	346	118	5	79	2.80
0.015	275	898	535	306	57	273	3.33
0.010	183	2,474	896	1,163	415	1,320	5.01

Result: 2,474 frequent itemsets (896 single products, 1,163 pairs, 371 triples, 43 of size 4, 1 of size 5) and **1,320 rules** (287 single-product -> single-product). Lift ranges from 4.5 to 77.0.

Top frequent itemsets with 2+ products:

items	support	baskets
JUMBO BAG PINK POLKADOT [22386] + JUMBO BAG RED RETROSPOT [85099B]	0.045	822
GREEN REGENCY TEACUP AND SAUCER [22697] + ROSES REGENCY TEACUP AND SAUCER [22699]	0.042	760
JUMBO STORAGE BAG SUKI [21931] + JUMBO BAG RED RETROSPOT [85099B]	0.039	711
JUMBO SHOPPER VINTAGE RED PAISLEY [22411] + JUMBO BAG RED RETROSPOT [85099B]	0.037	674
LUNCH BAG RED RETROSPOT [20725] + LUNCH BAG SUKI DESIGN [22383]	0.035	646
LUNCH BAG RED RETROSPOT [20725] + LUNCH BAG BLACK SKULL. [20727]	0.035	637
ALARM CLOCK BAKELIKE GREEN [22726] + ALARM CLOCK BAKELIKE RED [22727]	0.035	636
GREEN REGENCY TEACUP AND SAUCER [22697] + PINK REGENCY TEACUP AND SAUCER [22698]	0.034	629

Top frequent triples:

items	support	baskets
GREEN REGENCY TEACUP AND SAUCER [22697] + PINK REGENCY TEACUP AND SAUCER [22698] + ROSES REGENCY TEACUP AND SAUCER [22699]	0.029	535
JUMBO STORAGE BAG SUKI [21931] + JUMBO BAG PINK POLKADOT [22386] + JUMBO BAG RED RETROSPOT [85099B]	0.022	408
REGENCY CAKESTAND 3 TIER [22423] + GREEN REGENCY TEACUP AND SAUCER [22697] + ROSES REGENCY TEACUP AND SAUCER [22699]	0.022	402
JUMBO STORAGE BAG SUKI [21931] + JUMBO SHOPPER VINTAGE RED PAISLEY [22411] + JUMBO BAG RED RETROSPOT [85099B]	0.021	385
JUMBO BAG PINK POLKADOT [22386] + JUMBO SHOPPER VINTAGE RED PAISLEY [22411] + JUMBO BAG RED RETROSPOT [85099B]	0.021	375

Top single-product rules by lift:

antecedent	consequent	support	confidence	lift
HERB MARKER THYME [22916]	HERB MARKER ROSEMARY [22917]	0.012	0.932	71.25
HERB MARKER ROSEMARY [22917]	HERB MARKER THYME [22916]	0.012	0.921	71.25

HERB MARKER CHIVES [22921]	HERB MARKER PARSLEY [22918]	0.011	0.923	71.15
HERB MARKER PARSLEY [22918]	HERB MARKER CHIVES [22921]	0.011	0.810	71.15
HERB MARKER CHIVES [22921]	HERB MARKER THYME [22916]	0.010	0.904	69.96
HERB MARKER THYME [22916]	HERB MARKER CHIVES [22921]	0.010	0.797	69.96
HERB MARKER THYME [22916]	HERB MARKER PARSLEY [22918]	0.012	0.903	69.56
HERB MARKER PARSLEY [22918]	HERB MARKER THYME [22916]	0.012	0.899	69.56

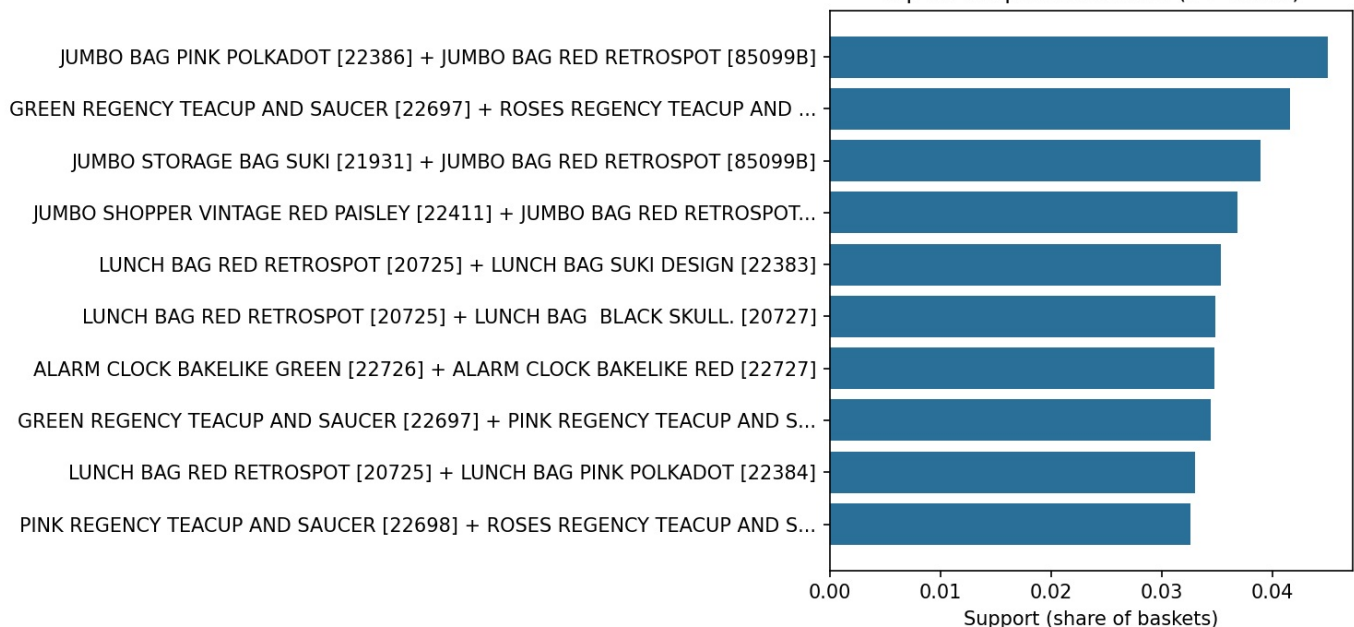
Highest-support rules (most commercially relevant):

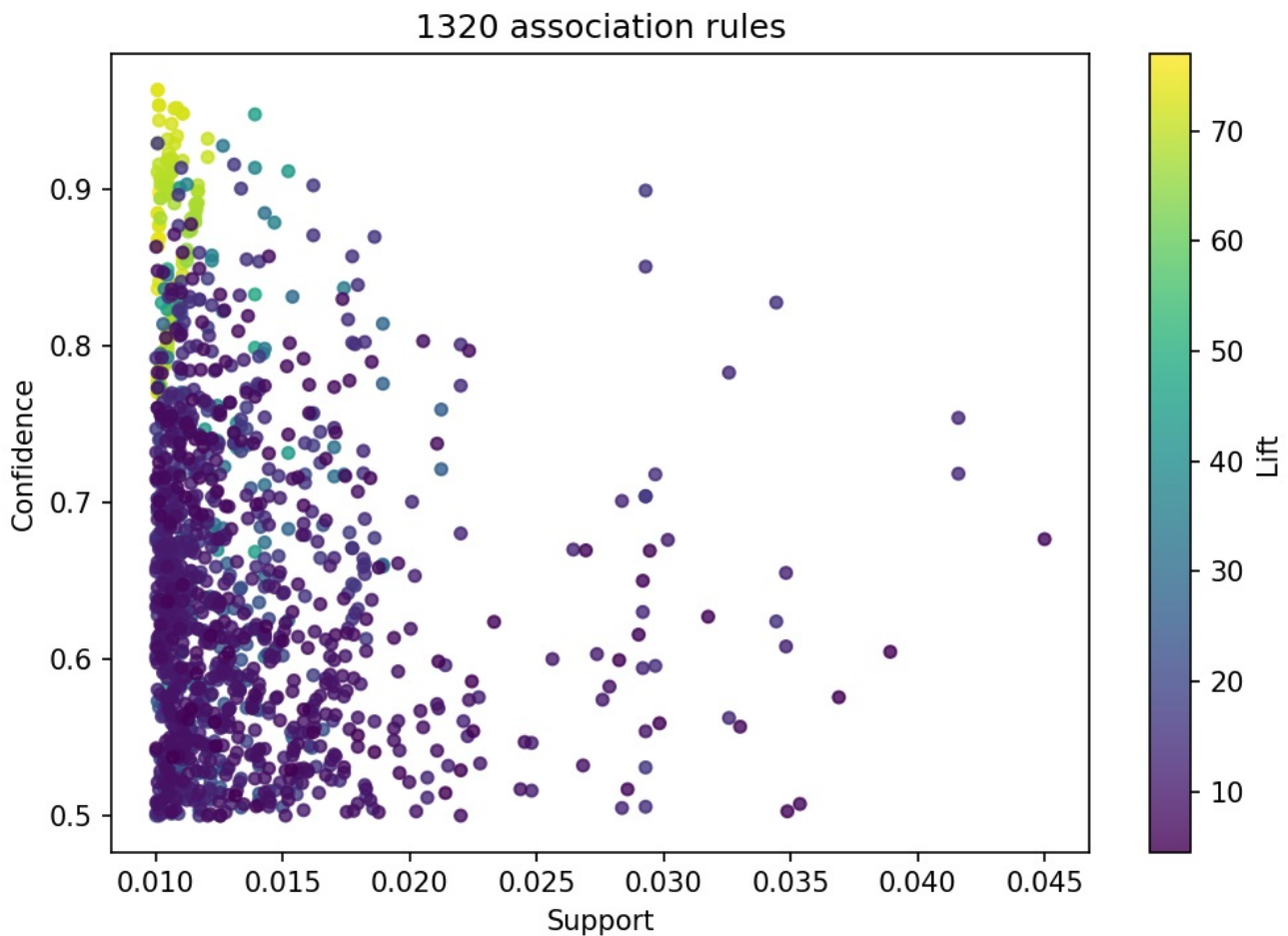
antecedent	consequent	support	confidence	lift
JUMBO BAG PINK POLKADOT [22386]	JUMBO BAG RED RETROSPOT [85099B]	0.045	0.677	5.98
ROSES REGENCY TEACUP AND SAUCER [22699]	GREEN REGENCY TEACUP AND SAUCER [22697]	0.042	0.718	13.02
GREEN REGENCY TEACUP AND SAUCER [22697]	ROSES REGENCY TEACUP AND SAUCER [22699]	0.042	0.754	13.02
JUMBO STORAGE BAG SUKI [21931]	JUMBO BAG RED RETROSPOT [85099B]	0.039	0.605	5.35
JUMBO SHOPPER VINTAGE RED PAISLEY [22411]	JUMBO BAG RED RETROSPOT [85099B]	0.037	0.576	5.09
LUNCH BAG SUKI DESIGN [22383]	LUNCH BAG RED RETROSPOT [20725]	0.035	0.507	5.96
LUNCH BAG BLACK SKULL. [20727]	LUNCH BAG RED RETROSPOT [20725]	0.035	0.503	5.90
ALARM CLOCK BAKELIKE RED [22727]	ALARM CLOCK BAKELIKE GREEN [22726]	0.035	0.608	11.44

Rules linking different product families (first two words of the name differ):

antecedent	consequent	support	confidence	lift
REGENCY SUGAR BOWL GREEN [23174]	REGENCY MILK JUG PINK [23175]	0.013	0.751	41.81
REGENCY MILK JUG PINK [23175]	REGENCY SUGAR BOWL GREEN [23174]	0.013	0.735	41.81
SET/10 BLUE POLKADOT PARTY CANDLES [21124]	SET/10 PINK POLKADOT PARTY CANDLES [21122]	0.010	0.774	41.22
SET/10 PINK POLKADOT PARTY CANDLES [21122]	SET/10 BLUE POLKADOT PARTY CANDLES [21124]	0.010	0.539	41.22
PINK POLKADOT CUP [21239]	BLUE POLKADOT CUP [21240]	0.011	0.670	37.89
BLUE POLKADOT CUP [21240]	PINK POLKADOT CUP [21239]	0.011	0.635	37.89

Top 10 frequent itemsets (2+ items)





Support = share of baskets containing both products; *confidence* = $P(\text{consequent} \mid \text{antecedent})$; *lift* = $\text{confidence} / P(\text{consequent})$ (lift 1 = independent).

Accuracy check. A from-scratch Apriori (`apriori/apriori_scratch.py`, tid-list bitsets) was run on the same data. It found the **same 2,474 itemsets and the same 1,320 rules**, with a maximum support/confidence/lift difference of 0.0. This cross-check also caught a real bug during development (a support denominator that ignored baskets without any frequent item), which is fixed. The library run takes about 4.97 s.

11. Outlier and noise analysis

Method: **Tukey IQR fences on the log10 scale** (values span orders of magnitude, so raw-scale fences flag 5-8% of ordinary values); *outlier* = beyond $1.5 \times \text{IQR}$, *extreme* = beyond $3 \times \text{IQR}$.

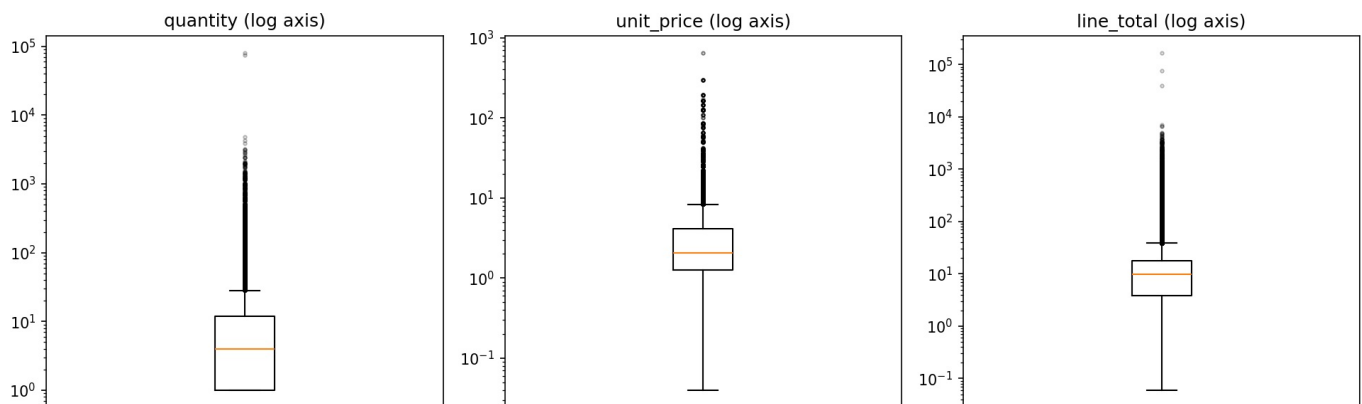
feature	k	lower_fence	upper_fence	outliers	pct
quantity	1.50	0.020	498.83	434	0.080
quantity	3.00	0.000	20,736.00	2	0.000
unit_price	1.50	0.210	24.80	4,855	0.930
unit_price	3.00	0.030	148.96	42	0.010
line_total	1.50	0.400	171.13	7,900	1.52
line_total	3.00	0.040	1,654.62	136	0.030

- Quantity: only **2** extreme lines - the 80,995-unit and 74,215-unit orders - and both were **cancelled minutes later** by a matching `c` invoice.
- Line value: 136 extreme lines; **10** of them were fully reversed by a later cancellation (same customer, product and quantity). These 10 lines, worth 311,623.96 (3.05% of revenue), are treated as **noise** and excluded from the value/customer calculations of the notebook. The other 126 extreme lines are valid wholesale orders and are **retained**.
- Unit price: genuine expensive items are retained.

feature	lower_fence	upper_fence	high_outliers	low_outliers	pct

monetary	24.31	20,232.68	38	12	1.15
frequency	0.090	55.90	12	0	0.280

- 38 high-spend customers (28.98% of identified-customer revenue) are **kept and treated as a separate VIP/wholesale segment**; removing them would remove the most valuable customers.
- Outliers are not deleted from the warehouse; Apriori uses only the *presence* of a product in a basket, so quantity outliers cannot influence it.



12. Similarity / dissimilarity analysis

Entities	Representation	Measure	Why
Customers (3,698 with ≥ 10 distinct products)	binary vector over products	Jaccard similarity (dissimilarity = $1 - \text{similarity}$)	asymmetric binary data: shared absences carry no information
Products (those in $\geq 1\%$ of baskets)	binary vector over baskets	cosine similarity	compares co-occurrence patterns, corrects for popularity
Customers	standardised log-RFM (spend, frequency, recency)	Euclidean distance	numeric features on a common scale

All three implementations were validated against `scipy` / the textbook formula on 200 random pairs (assertions in the notebook).

- **Jaccard**: mean similarity 0.019, median 0.013; **33.58%** of the 6,835,753 customer pairs share no product at all (99th percentile 0.097).

customer_a	customer_b	jaccard_similarity	jaccard_dissimilarity	shared_products	products_a	products_b
14,567	14,937	1.00	0.000	33	33	33
14,567	17,173	0.816	0.184	31	33	36
14,937	17,173	0.816	0.184	31	33	36
14,937	18,167	0.727	0.273	24	33	24
14,567	18,167	0.727	0.273	24	33	24
17,173	18,167	0.667	0.333	24	36	24

- **Cosine (nearest products):**

product	most_similar_product	cosine
WHITE HANGING HEART T-LIGHT HOLDER [85123A]	RED HANGING HEART T-LIGHT HOLDER [21733]	0.384
WHITE HANGING HEART T-LIGHT HOLDER [85123A]	CANDLEHOLDER PINK HANGING HEART [22804]	0.328
JUMBO BAG RED RETROSPOT [85099B]	JUMBO BAG PINK POLKADOT [22386]	0.519
JUMBO BAG RED RETROSPOT [85099B]	JUMBO STORAGE BAG SUKI [21931]	0.456
REGENCY CAKESTAND 3 TIER [22423]	ROSES REGENCY TEACUP AND SAUCER [22699]	0.361
REGENCY CAKESTAND 3 TIER [22423]	GREEN REGENCY TEACUP AND SAUCER [22697]	0.357
PARTY BUNTING [47566]	SPOTTY BUNTING [23298]	0.347
PARTY BUNTING [47566]	TEA TIME PARTY BUNTING [47566B]	0.232
LUNCH BAG RED RETROSPOT [20725]	LUNCH BAG PINK POLKADOT [22384]	0.465
LUNCH BAG RED RETROSPOT [20725]	LUNCH BAG SUKI DESIGN [22383]	0.459

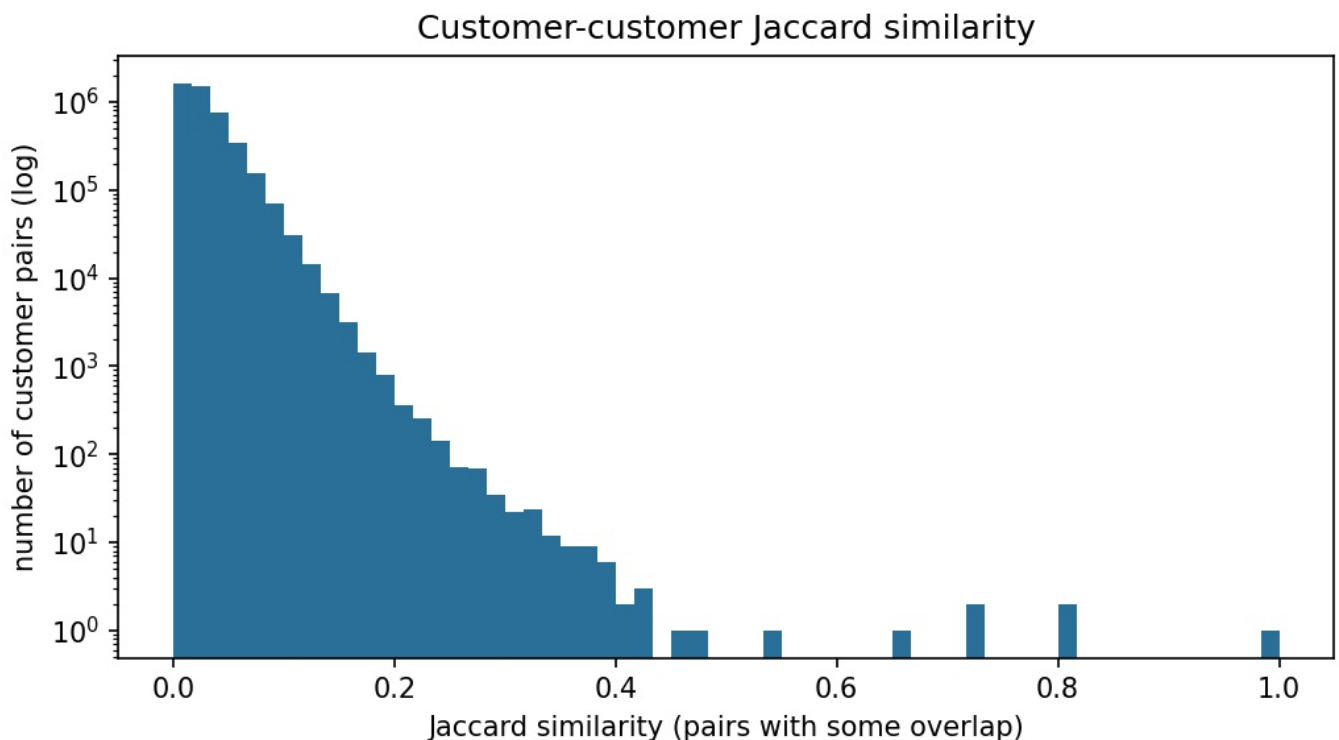
Across the 287 single-product rules, cosine similarity and lift are rank-correlated (Spearman = 0.870), so the two

methods agree.

- **Euclidean (RFM)** for the top spender (customer 14646):

role	customer_id	euclidean_distance	log_monetary	log_frequency	recency_days
reference (top spender)	14,646	0.000	5.44	1.86	1
most similar	18,102	0.210	5.41	1.78	0
most similar	17,450	0.586	5.28	1.66	8
most similar	14,156	0.770	5.06	1.73	9
most dissimilar	16,738	10.55	0.570	0.000	297
most dissimilar	16,446	10.51	0.460	0.000	205
most dissimilar	16,454	9.83	0.770	0.000	64

- Customer similarity by purchased products (Jaccard) and by RFM distance are almost unrelated (Spearman = -0.053): customers of similar *value* do not necessarily buy the *same products*.



13. Key findings

1. **Cleaning matters:** 3.6% of the raw rows are not valid product sales (duplicates, cancellations, fees, zero-price rows).
2. **One market:** the United Kingdom brings 85.17% of revenue; the 4,334 identified customers are concentrated - the 386 customers with 10+ invoices produce 51.1% of identified-customer revenue.
3. **Seasonality:** revenue peaks in 2011-11 (1,452,113), consistent with the gift/Christmas season (December 2011 is incomplete: the data ends on 9 December).
4. **Co-purchase is dominated by product families:** the strongest rules connect colour/design variants and set pieces (herb markers, Regency tea set, jumbo bags, lunch bags, party candles, bunting), not unrelated products.
5. **Outliers need context:** the two most extreme orders were cancelled, so a naive "top customer" ranking would have been wrong; real high-value customers are a small group that earns a large share of revenue.

14. Business and purchasing insights

- **Sell sets, not single items.** HERB MARKER THYME -> HERB MARKER ROSEMARY has confidence 93% and lift 71: 93% of the baskets containing HERB MARKER THYME also contain HERB MARKER ROSEMARY. Bundle the herb markers as a set and show the remaining variants as soon as one is in the basket.
- **Cross-sell within a collection.** The Regency tea set is the clearest case: GREEN and ROSES teacups appear

together in 4.2% of baskets (confidence 75%, lift 13), and the triple of green, pink and roses cups appears in 2.9% of baskets. The rule REGENCY SUGAR BOWL GREEN -> REGENCY TEAPOT ROSES (confidence 72%, lift 31.8) shows customers completing a tea service across different piece types.

- **Jumbo bags are an anchor product:** JUMBO BAG RED RETROSPOT is the second most frequent product and the consequent of several high-support rules (e.g. JUMBO STORAGE BAG SUKI -> JUMBO BAG RED RETROSPOT: support 3.9%, confidence 60%, lift 5.3), so it is the natural anchor for recommendations and promotions.
- **Treat wholesale customers separately.** A small VIP segment earns about 29% of identified-customer revenue; they need account management, not the same campaigns as one-time buyers (who make up 1,505 of the identified customers).
- **Recommendations:** use product-to-product cosine/Apriori for "customers also bought"; use RFM distance to find customers who look like the VIPs.

15. Limitations

- The AWS parts (Glue, Athena, Redshift) were **written to AWS specifications but not run on AWS** in the development environment. The Glue jobs ran on local PySpark with identical code (the `awsglue` import is optional), the SQL was validated on DuckDB with the same schema, and the Redshift DDL/`COPY` and deployment script are untested.
- The CDC updates and deletes are **synthetic**; the snapshots model how a change feed behaves, not real business events. The current table therefore differs slightly from the cleaned data (1,542 deleted lines, 3,831 corrected quantities).
- Cancellations are excluded rather than netted against the original sale; the noise rule only catches reversals with an exact quantity match.
- 131,354 clean rows (25.1%) have no `CustomerID`: they are used for baskets and revenue but not for customer analysis.
- Apriori works at `StockCode` level, so colour variants are separate items; results are for one retailer over about one year; many rules appear in mirrored pairs (A -> B and B -> A).
- The dataset is one year of data; seasonality cannot be separated from trend.

16. Conclusion

The pipeline covers every step of the assignment as one flow: Glue turns 541,909 raw rows into 522,504 validated sales lines with a full audit trail, a snapshot-based CDC simulation derives and replays inserts, updates and deletes (replay verified against v3), Athena and Redshift expose the same current table, Apriori finds 1,320 rules (verified by an independent implementation), and outlier/similarity analysis separates genuine high-value customers from cancelled-order noise. The main business result is that customers buy **collections and colour variants together**, which supports bundling and in-basket recommendations; the main caveat is that the AWS deployment still has to be executed in an AWS account following `docs/aws_deployment.md`.